

Rule-Based AI Chatbot

Organization: Decode Labs

Submitted By: Shah Mubarak Zaib

Internship Domain: Artificial Intelligence

Programming Language: Python

1. Introduction

Artificial Intelligence (AI) is a branch of computer science that enables machines to perform tasks that normally require human intelligence. One of the simplest applications of AI is a **Rule-Based Chatbot**, which responds to users based on predefined rules rather than learning from data.

This project demonstrates the implementation of a Rule-Based AI Chatbot using Python. The chatbot interacts with users by recognizing predefined inputs and providing appropriate responses. It serves as an introduction to AI concepts such as decision-making, control flow, user interaction, and basic natural language processing techniques like input normalization.

2. Project Objective

The objective of this project is to develop a simple Rule-Based AI Chatbot capable of:

- Accepting user input.
- Responding to predefined questions and greetings.
- Running continuously until the user exits.
- Using Python dictionaries for efficient response management.
- Demonstrating the fundamentals of AI through rule-based decision making.

3. Planning the Chatbot

Before writing any code, I identified the main components required for the chatbot.

The chatbot should be able to:

- Display a welcome message.
- Accept user input.
- Process the user's input.

- Match the input with predefined responses.
- Display the appropriate response.
- Continue the conversation.
- Exit when requested by the user.

This planning stage helped organize the program into smaller, manageable sections.

4. Technologies Used

Technology	Purpose
Python 3	Programming Language
Dictionary	Store chatbot responses
While Loop	Continuous interaction
Input/Output Functions	User communication
If Statement	Exit condition
String Methods (<code>lower()</code> , <code>strip()</code>)	Input normalization

5. Displaying the Welcome Message

The first part of the program displays a welcome message using the `print()` function.

The welcome screen informs the user about the chatbot and explains how to end the conversation by typing **bye**, **exit**, or **quit**.

This improves the user experience by providing clear instructions before the interaction begins.

6. Creating the Knowledge Base

The chatbot's knowledge was stored in a Python dictionary named `responses`.

Each dictionary entry consists of:

- **Key:** User's question or command.
- **Value:** Chatbot's response.

Example:

```
responses = {  
    "hello": "Hi! How can I help you today?",  
    "what is ai": "AI stands for Artificial Intelligence."  
}
```

Using a dictionary makes the program cleaner and easier to maintain compared to writing many if-elif statements.

7. Creating a Continuous Conversation

To allow the chatbot to keep interacting with the user, I used an infinite loop.

`while True:`

This loop keeps the chatbot running until the user decides to exit.

Without this loop, the chatbot would answer only one question before terminating.

8. Accepting User Input

Inside the loop, I used the `input()` function to receive messages from the user.

```
user = input("You: ")
```

The user's message is stored in the variable `user` for further processing.

9. Processing the Input

To ensure consistent input handling, I normalized the user's message using:

```
user = input("You: ").lower().strip()
```

Purpose

- `lower()` converts all letters to lowercase.
- `strip()` removes unnecessary spaces from the beginning and end of the input.

For example:

Input:
HeLLo

Processed:
hello

This allows the chatbot to recognize the same command regardless of capitalization or extra spaces.

10. Implementing the Exit Condition

To end the conversation politely, I added an exit condition.

```
if user in ["bye", "exit", "quit"]:
    print("Bot: Goodbye!")
    break
```

When the user enters any of these commands, the chatbot displays a farewell message and exits the loop using the break statement.

11. Retrieving Response

After processing the input, the chatbot searches for the user's message inside the dictionary.

```
response = responses.get(
    user,
    "Sorry, I don't understand that."
)
```

The get() method searches for the entered key.

- If the key exists, the corresponding response is returned.
- If the key does not exist, the chatbot returns a default message.

This approach prevents errors and ensures that the chatbot always responds.

12. Displaying the Response

Finally, the chatbot displays the retrieved response using:

```
print("Bot:", response)
```

The chatbot then waits for the next user input, allowing continuous interaction.

13. Testing the Chatbot

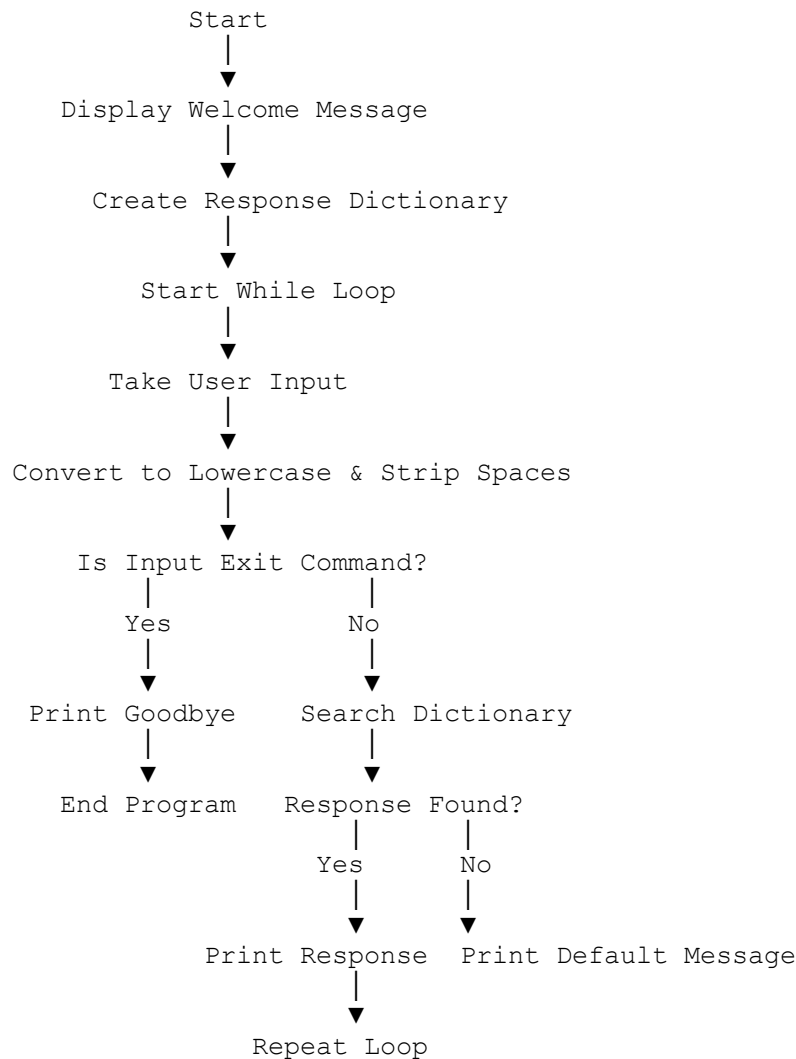
After completing the implementation, I tested the chatbot with different types of inputs.

The chatbot was tested for:

- Greeting messages.
- AI-related questions.
- Unknown questions.
- Uppercase and lowercase input.
- Extra spaces.
- Exit commands.

All test cases produced the expected results.

14. Flowchart



15. Challenges Faced

During development, I focused on making the chatbot more user-friendly.

The main challenges included:

- Handling different capitalizations.
- Ignoring extra spaces.
- Avoiding long chains of if-elif statements.
- Ensuring the chatbot never crashed on unknown input.

These challenges were solved using string methods, dictionaries, and Python's `get()` function.

16. Final Output

The completed chatbot successfully:

- Displays a welcome message.
- Accepts user input continuously.
- Uses rule-based decision making.
- Responds to predefined questions.
- Handles unknown input gracefully.
- Ends the conversation when requested.

The project demonstrates the basic principles of Artificial Intelligence using logical decision-making and Python programming.

17. Sample Execution

Welcome to the Rule-Based AI Chatbot!

You: hello

Bot: Hi! How can I help you today?

You: what is ai

Bot: AI stands for Artificial Intelligence.

You: how are you

Bot: I'm doing great! Thanks for asking.

You: thanks

Bot: My pleasure!

You: what is java

Bot: Sorry, I don't understand that. Please try another question.

You: bye

Bot: Goodbye! Have a great day.

18. Conclusion

This project provided practical experience in developing a Rule-Based AI Chatbot using Python. Through this implementation, I gained a better understanding of control flow, loops, dictionaries, input processing, and decision-making logic. Although the chatbot is based on predefined rules, it establishes a strong foundation for learning more advanced AI concepts such as Machine Learning, Natural Language Processing, and intelligent conversational systems.

.

Artificial Intelligence Internship – Project 1